

Graph Databases and Neo4j

Chapter 06 - Sub-Chapter 06

NoSQL Databases Series

What is a Graph DB?	Nodes, relationships, properties explained
Neo4j and Cypher	Pattern matching queries -- simpler than SQL JOINS
Spring Data Neo4j	@Node, @Relationship, repositories
Use Cases	Fraud detection, recommendations, knowledge graphs

1

What is a Graph Database?

Nodes, relationships, and the power of connections

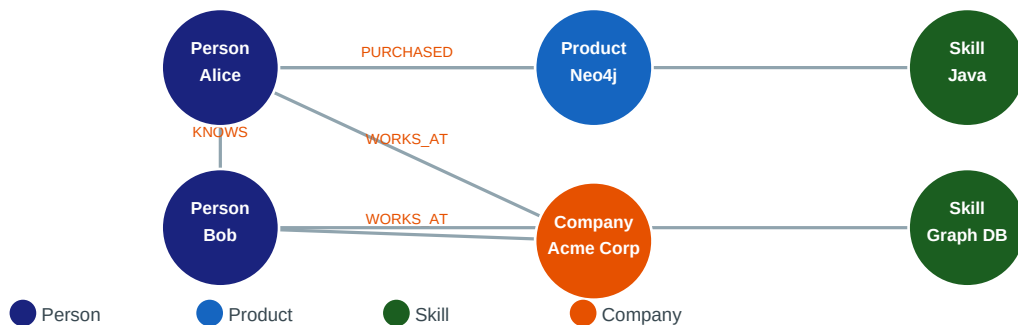
Imagine trying to find all friends-of-friends of a user in a social network using SQL. You would write five JOINS, and at depth 3 or 4, the query becomes impractical. A graph database is purpose-built for exactly this kind of connected data.

A graph database stores data as **nodes** (entities) and **relationships** (connections). Every relationship is a first-class citizen — stored with its own properties, directly connecting two nodes. Traversing a relationship is a single pointer follow, not a JOIN.

The three building blocks:

- **Node** — an entity. Has labels (type) and properties. E.g., *(Person {name:'Alice', age:30})*
- **Relationship** — a directed, named connection between two nodes. E.g., *-[:KNOWS]->*
- **Property** — a key-value pair on a node or relationship. E.g., *{since: 2020}*

Graph Database — Nodes, Relationships, and Properties



Why relationships are stored differently from SQL: In a relational database, a JOIN computes connections at query time by scanning foreign keys. In a graph database, relationships are physically stored as pointers — traversing a million relationships takes the same time as traversing one.

2 Neo4j and Cypher Query Language

The most popular graph database and its query language

Neo4j is the world's leading graph database, created in 2007 in Sweden. It comes with Cypher — a declarative query language designed to look like ASCII art of the graph pattern you want to match. Once it clicks, it is far more intuitive than SQL for connected queries.

Cypher vs SQL — Same Question, Different Syntax

SQL: Friends of Alice who like Neo4j

```
SELECT p2.name
FROM persons p1
JOIN friendships f ON p1.id=f.from_id
JOIN persons p2 ON p2.id=f.to_id
JOIN likes l ON p2.id=l.person_id
JOIN products pr ON pr.id=l.product_id
WHERE p1.name='Alice' AND pr.name='Neo4j'
```

Cypher: Same question, 2 lines

```
MATCH (a:Person {name:'Alice'})
      -[:KNOWS]->(friend:Person)
      -[:PURCHASED]->(p:Product {name:'Neo4j'})
RETURN friend.name
```

Cypher basics — the pattern language:

```
// Nodes use parentheses: (n:Label {property: value})
// Relationships use arrows: -[:TYPE {property: value}]->
// Pattern: (node1)-[:RELATIONSHIP]->(node2)

// CREATE -- add nodes and relationships
CREATE (alice:Person {name: 'Alice', age: 30, city: 'New York'})
CREATE (bob:Person {name: 'Bob', age: 28, city: 'London'})
CREATE (neo4j:Product {name: 'Neo4j', category: 'Database'})

// Create relationships
MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Bob'})
CREATE (a)-[:KNOWS {since: 2020, strength: 'close'}]->(b)

MATCH (a:Person {name: 'Alice'}), (p:Product {name: 'Neo4j'})
CREATE (a)-[:PURCHASED {date: date('2024-01-15'), amount: 299}]->(p)
```

MATCH — pattern-based queries:

```
// Find all friends of Alice
MATCH (alice:Person {name: 'Alice'})-[:KNOWS]->(friend:Person)
RETURN friend.name, friend.city

// Friends of friends (2 hops -- expensive in SQL, trivial in graph)
MATCH (alice:Person {name: 'Alice'})-[:KNOWS*2]->(fof:Person)
WHERE fof.name <> 'Alice'
RETURN DISTINCT fof.name

// Variable-length path: friends up to 3 hops away
MATCH (alice:Person {name: 'Alice'})-[:KNOWS*1..3]->(person:Person)
RETURN DISTINCT person.name, length(path) AS hops
ORDER BY hops

// Who purchased the same product as Alice?
MATCH (alice:Person {name: 'Alice'})-[:PURCHASED]->(p:Product)
      <-[:PURCHASED]-(other:Person)
WHERE other.name <> 'Alice'
RETURN other.name, p.name AS commonProduct
```

Aggregation and advanced Cypher:

```
// Count friends per person, find most connected
MATCH (p:Person)-[:KNOWS]->(friend:Person)
RETURN p.name, COUNT(friend) AS friendCount
ORDER BY friendCount DESC
LIMIT 10

// Shortest path between two people
MATCH path = shortestPath(
  (alice:Person {name: 'Alice'})-[:KNOWS*]-(bob:Person {name: 'Bob'})
)
RETURN path, length(path) AS degrees

// Page Rank -- who is most influential? (built-in graph algorithm)
CALL gds.pageRank.stream('social-graph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS person, score
ORDER BY score DESC LIMIT 10
```

3 Spring Data Neo4j

Nodes as Java classes, relationships as fields

Spring Data Neo4j (SDN) maps graph nodes to Java classes and relationships to fields, just as JPA maps SQL tables to entities. You get repositories, query derivation from method names, and `@Query` for custom Cypher.

Maven dependency:

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-neo4j</artifactId>
</dependency>
```

application.yml:

```
spring:
  neo4j:
    uri: bolt://localhost:7687          # Bolt protocol
    authentication:
      username: neo4j
      password: secret
    data:
      neo4j:
        database: neo4j                # target database
```

Node entity class:

```
import org.springframework.data.neo4j.core.schema.*;
import java.util.List;
import java.util.Set;

@Node("Person")                                // maps to Neo4j node with label 'Person'
public class Person {

    @Id @GeneratedValue
    private Long id;

    private String name;
    private int age;
    private String city;

    @Relationship(type = "KNOWS", direction = Relationship.Direction.OUTGOING)
    private List<Person> friends;                // traverses KNOWS relationships

    @Relationship(type = "PURCHASED")
    private Set<Product> purchases;              // traverses PURCHASED relationships

    // getters/setters
}

@Node("Product")
public class Product {
    @Id @GeneratedValue
    private Long id;
    private String name;
    private String category;
    // getters/setters
}
```

```
}
```

Repository and custom Cypher:

```
import org.springframework.data.neo4j.repository.Neo4jRepository;
import org.springframework.data.neo4j.repository.query.Query;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface PersonRepository extends Neo4jRepository<Person, Long> {

    // Derived query from method name
    List<Person> findByCity(String city);

    // Custom Cypher query
    @Query("MATCH (p:Person {name: $name})-[:KNOWS*1..2]->(friend:Person) " +
        "RETURN DISTINCT friend")
    List<Person> findFriendsOfFriends(String name);

    // Find influencers: people with most followers
    @Query("MATCH (p:Person)-[:KNOWS]-(follower:Person) " +
        "RETURN p ORDER BY COUNT(follower) DESC LIMIT $limit")
    List<Person> findTopInfluencers(int limit);

    // Who is connected to both alice and bob?
    @Query("MATCH (a:Person {name:'Alice'})-[:KNOWS]-(mutual:Person)-[:KNOWS]-(b:Person {name:'Bob' " +
        '}) " +
        "RETURN mutual")
    List<Person> findMutualConnections();
}
```

Service using Neo4jTemplate for advanced queries:

```
import org.springframework.data.neo4j.core.Neo4jTemplate;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Map;

@Service
public class RecommendationService {

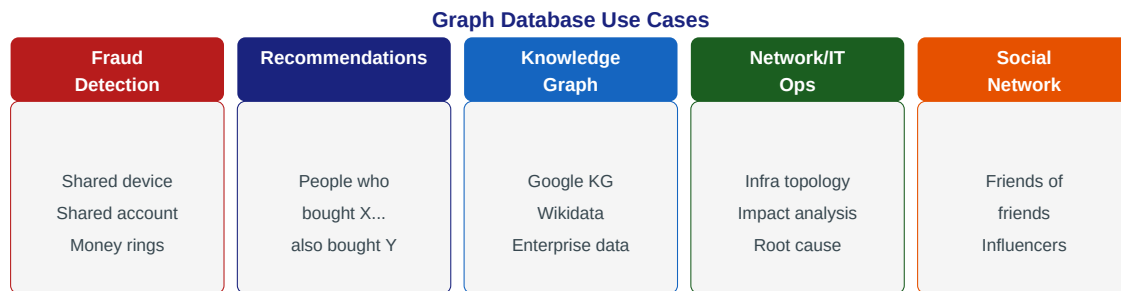
    private final Neo4jTemplate neo4jTemplate;

    public RecommendationService(Neo4jTemplate neo4jTemplate) {
        this.neo4jTemplate = neo4jTemplate;
    }

    // Collaborative filtering: "Users like you also bought..."
    public List<Map<String, Object>> recommendProducts(String userId) {
        String cypher =
            "MATCH (me:Person {id: $userId})-[:PURCHASED]->(p:Product)" +
            "    <-[:PURCHASED]-(similar:Person)" +
            "    -[:PURCHASED]->(recommended:Product) " +
            "WHERE NOT (me)-[:PURCHASED]->(recommended) " +
            "RETURN recommended.name AS product, COUNT(similar) AS score " +
            "ORDER BY score DESC LIMIT 5";
        return neo4jTemplate.findAll(cypher, Map.of("userId", userId), Map.class);
    }
}
```

4 Graph Database Use Cases

Where graphs outperform all other databases



Fraud Detection — the killer use case:

Fraudsters reuse devices, email addresses, and phone numbers across fake accounts. In a relational database, detecting account rings requires complex recursive CTEs. In Neo4j, it is a single pattern match:

```
// Find accounts sharing a device -- fraud ring detection
MATCH (account1:Account)-[:USED_DEVICE]->(device:Device)
      <-[:USED_DEVICE]-(account2:Account)
WHERE account1 <> account2
WITH device, COLLECT(DISTINCT account1) + COLLECT(DISTINCT account2) AS ring
WHERE SIZE(ring) > 3           // rings of 4+ accounts are suspicious
RETURN device.id AS sharedDevice, SIZE(ring) AS ringSize, ring
ORDER BY ringSize DESC

// This query runs in milliseconds on millions of accounts
// The SQL equivalent with recursive CTEs would take minutes
```

Knowledge Graph — connecting enterprise data:

Google's Knowledge Graph connects 70+ billion facts. Neo4j is used at companies like NASA, Walmart, and Airbus to connect disparate data sources into a unified knowledge graph that answers questions no single SQL query could:

```
// What components could be affected if supplier X fails?
MATCH (supplier:Supplier {name: 'Acme Parts'})
      -[:SUPPLIES]->(component:Component)
      -[:USED_IN*1..5]->(product:Product)
RETURN DISTINCT product.name, component.name, supplier.name

// Impact analysis: if this microservice goes down, what breaks?
MATCH (svc:Service {name: 'payments-api'})
      <-[:DEPENDS_ON*]-(affected:Service)
RETURN DISTINCT affected.name, affected.team
```

When NOT to use a graph database:

x Simple CRUD with no connections: A user table with no relationships — use PostgreSQL.

x Time-series data: Sensor readings, logs — use InfluxDB or Cassandra.

x Full-text search dominated workloads: Use Elasticsearch.

x Very high write throughput: Graph consistency constraints make high-volume writes slower than Cassandra or DynamoDB.

Key Rules

- Use graph databases when relationships are as important as the data itself.
- Neo4j Cypher pattern matching reads like English -- (Person)-[:KNOWS]->(Person).
- Graph algorithms (PageRank, community detection) run natively on the data -- no ETL.
- Start with Neo4j Aura Free (cloud) to prototype -- no local install needed.
- Combine with relational DB: use Postgres for transactions, Neo4j for recommendation/fraud queries.